

Executive assessment

The project has a strong core idea, but the current proposal is **too ambitious, partially inconsistent, and not yet implemented**.

The most promising direction is not merely:

“Use a log-polar grid because nearby objects require more resolution.”

A stronger and more defensible solution is:

A planner-aware, uncertainty-aware, multi-surface adaptive 2.5D map that spends resolution where collision risk and information uncertainty are highest—not simply where objects are closest.

I would position this as **RiskFovea-MLS** or **SaarthiMap**:

- **RiskFovea**: resolution follows risk, uncertainty, and the planned trajectory.
- **MLS**: each cell can retain two or three vertical surface hypotheses instead of only one height.
- **Safety-aware**: explicitly distinguishes occupied, free, occluded, and unknown space.
- **Edge-aware**: automatically degrades gracefully when computation or sensor quality falls.

1. What the problem is really asking

From the supplied research documents, the intended system is:

1. Receive raw LiDAR point clouds.
2. Remove noise and compensate for vehicle motion.
3. Identify terrain, static obstacles, and potentially dynamic objects.
4. Compress the 3D cloud into a lightweight 2.5D map.
5. Use high resolution where detail matters and coarse resolution elsewhere.
6. update the map temporally without leaving “ghost” obstacles.
7. Run in real time on edge hardware.
8. Demonstrate memory, latency, and accuracy improvements.

The proposed cell currently stores approximately:

```
ground elevation
maximum obstacle height
dominant semantic class
occupancy confidence
```

This appears explicitly in the research document.

The intended evaluation includes segmentation accuracy, latency, VRAM use, and comparison against uniform-resolution maps.

Important limitation

I could not find the **original official SIH problem-statement document** in the workspace. The available files are interpretations and research reports. Therefore, claims such as:

- deployment specifically in Vallichira,
- DRDO as the organization,
- mandatory Jetson AGX Orin use,
- exact FPS or memory requirements,

should be checked against the official SIH statement before presenting them as requirements.

2. Current project status

At present, there is effectively **no implementation**.

The notebook only contains Kaggle boilerplate that:

- imports NumPy, pandas, `os`, and `kagglehub` ;
- recursively prints the contents of the mounted nuScenes-mini dataset.

It does not currently:

- load a LiDAR scan,
- read semantic labels,
- create a grid,
- run segmentation,
- perform temporal mapping,
- visualize results,
- or calculate metrics.

The two useful documents are:

- the ambitious technical research proposal:
- the more realistic beginner/hackathon guide:

The `deep-research-report.md` file is unrelated to LiDAR and should not be treated as part of the solution.

3. Problems in the current proposal

3.1 Distance-only foveation is not enough

The proposal assumes:

- nearby area → high resolution,
- distant area → low resolution.

That is a reasonable baseline, but not always safe.

Examples:

- A flat road two metres away does not need five-centimetre detail everywhere.
- A pedestrian 40 metres ahead on the vehicle's projected trajectory may require finer resolution.
- A nearby wall behind the vehicle may be less important than an uncertain ditch 20 metres ahead.
- A fast oncoming object may be more important than a closer parked vehicle.

Therefore, distance should only be the **base resolution prior**. Refinement should also depend on:

- time to collision,
- planned path,
- relative velocity,
- terrain roughness,
- semantic uncertainty,
- temporal change,
- and sensor confidence.

This is the most important opportunity to make the project genuinely innovative.

3.2 A single-height 2.5D cell loses critical structure

The current proposal keeps ground height and maximum obstacle height. That still cannot correctly represent:

- road underneath a bridge,
- a tree branch over a traversable road,
- vegetation with empty space below it,
- a tunnel,
- stacked roads,
- or a culvert.

The provided paper is specifically about **layered 2.5D maps for multilevel environments**, not a single height per XY location.

The current map is therefore misaligned with one of its strongest supporting references.

Better approach

Retain up to $\kappa = 2$ or $\kappa = 3$ vertical surface hypotheses per cell:

Surface 1: ground
Surface 2: obstacle/canopy/bridge
Surface 3: optional additional layer

Each hypothesis can contain:

```

z_min, z_mean, z_max
surface normal or slope
roughness/variance
semantic probabilities
occupied/free/unknown evidence
last observed timestamp

```

This remains much cheaper than a full 3D voxel map while handling overhangs far better.

3.3 Ground removal must not remove ground understanding

The report recommends running Patchwork++ first and removing 50–70% of points before neural-network inference.

That improves speed, but blindly stripping ground creates another problem: the most important hazards in unstructured terrain are often part of the ground:

- potholes,
- ditches,
- slopes,
- broken road edges,
- mud,
- puddles,
- loose gravel,
- and sudden elevation changes.

Better design: dual-path terrain processing

```

Ground points
└─ lightweight geometry path
    slope + roughness + height discontinuity + negative-obstacle analysis

Non-ground points
└─ semantic network
    vehicle + pedestrian + vegetation + wall + pole + other

```

Patchwork++ should provide a **ground prior**, not a command to discard ground points.

This hybrid architecture is more credible than asking one expensive network to do everything.

3.4 Semantic class is not the same as motion

The report proposes applying rapid decay to classes such as vehicle and pedestrian.

That creates several errors:

- a parked car is semantically a vehicle but is static;
- moving vegetation is semantically vegetation but may be dynamic;
- a fallen object might move but belong to a nominally static class;

- temporary occlusion could cause a real obstacle to disappear too quickly.

Better approach

Estimate motion from temporally aligned observations:

```
motion score =
  occupancy change
+ centroid displacement
+ temporal residual after ego-motion compensation
+ semantic motion prior
```

Semantics should be a **prior**, not the final dynamic/static decision.

A cheap cell-level alpha-beta or Kalman filter is enough for a prototype. Full 3D bounding-box tracking is unnecessary.

3.5 “Not observed” is not the same as “free”

The current design talks mainly about occupied cells and confidence decay. But a safe map must distinguish:

1. **Occupied** — LiDAR returned from something.
2. **Free** — a LiDAR ray demonstrably travelled through the region.
3. **Unknown** — it was not observed.
4. **Occluded** — something blocked the sensor from seeing it.

Without this distinction, the map may mark an unseen area behind a truck or vegetation as safe.

Recommendation

Use ray traversal:

- cells before a valid return receive free-space evidence;
- the hit cell receives occupied evidence;
- cells behind the hit remain unknown/occluded;
- weather-degraded sectors get increased uncertainty.

This improvement may contribute more to navigation safety than increasing semantic mIoU by a few percentage points.

3.6 The log-polar claims are overstated

The report says log-polar mapping eliminates boundary artifacts and provides direct branch-free indexing.

That is not fully accurate.

Log-polar grids still have:

- a singularity near $r = 0$,
- an angular seam at $-\pi/\pi$,
- quantization boundaries,

- large angular cell widths at long ranges,
- distorted cell aspect ratios,
- `atan2` and logarithm computation costs.

It is a useful base grid, but not a complete solution to projection error.

More defensible claim

“A sensor-aligned polar or log-polar base lattice offers deterministic indexing and naturally approximates LiDAR sampling density, while sparse refinement patches preserve safety-critical detail.”

That is accurate and still sounds innovative.

3.7 Memory claims need honest accounting

The document describes a uniform five-centimetre grid over 200×200 metres as 16 million cells.

That count is correct for a square map, but 16 million cells do not automatically cause out-of-memory:

- one `float32` per cell: approximately 64 MB;
- 16 bytes per cell: approximately 256 MB;
- 32 bytes per cell: approximately 512 MB;
- multiple temporal buffers and semantic vectors increase this further.

The stronger argument is not “uniform grids always cause OOM.” It is:

Uniform grids waste memory bandwidth and update work on irrelevant or unobserved regions, reducing real-time performance and leaving less capacity for temporal, semantic, and uncertainty layers.

Benchmark real bytes and update latency instead of making an absolute OOM claim.

4. Recommended innovative solution: RiskFovea-MLS

4.1 Core architecture



4.2 Adaptive resolution based on utility

Start with a ring or log-polar base map. Then calculate a utility score for each region:

$$[U = w_pP + w_tT + w_uH + w_gG + w_mM + w_dD]$$

Where:

- P : closeness to the planned path;
- T : inverse time-to-collision;
- H : semantic or occupancy entropy;
- G : terrain roughness or elevation discontinuity;
- M : temporal motion score;
- D : distance prior.

Then:

- split high-utility cells;
- merge low-utility neighbouring cells;
- enforce a strict cell/memory budget;
- prevent refinement beyond actual sensor point density.

Why this is better

This lets you say:

“Our system does not simply look more closely near the vehicle. It looks more closely where the vehicle is likely to need information.”

That is a much stronger innovation statement.

4.3 Planned-path “attention corridor”

A particularly feasible version is to reserve high resolution in:

1. a small hard-safety radius around the vehicle;
2. a corridor surrounding the currently planned trajectory;
3. regions containing high uncertainty or approaching objects.

For example:

```
0-6 m safety envelope: never coarser than 10 cm
planned-path corridor: 10-20 cm
high-uncertainty patches: locally refine
background: 30-100 cm
```

When the vehicle turns, the high-resolution corridor moves with the predicted path.

This produces an excellent visual demo: the map appears to “look into” a turn before the vehicle reaches it.

4.4 Multi-surface adaptive cell

Each active cell could store:

```
Cell {
  terrain_surface
  upper_surface
  obstacle_height
  overhead_clearance
  slope
  roughness
  semantic_distribution
  occupied_evidence
  free_evidence
  unknown_evidence
  velocity
  timestamp
}
```

Keep only two surfaces for the hackathon prototype.

A gap-based clustering rule is sufficient:

```
Sort points by z.
Start a new surface if the vertical gap exceeds a threshold.
Keep the two strongest clusters.
```

This is much more feasible than full 3D voxelization.

4.5 Negative-obstacle detection

Most teams will demonstrate cars and pedestrians. A more distinctive off-road contribution would be detecting:

- potholes,
- ditches,
- broken road edges,
- and sudden drop-offs.

These are “negative obstacles”: the danger comes from missing ground rather than an object protruding upward.

Potential signals:

- expected ground continuation suddenly disappears;
- neighbouring ground heights exhibit a sharp negative step;
- ray pattern indicates a shadow region followed by a lower surface;
- local slope or height residual exceeds a vehicle-specific threshold.

A judge is more likely to remember:

“This system detects potholes and ditches that ordinary occupancy maps miss”

than another generic vehicle-segmentation demo.

4.6 Vehicle-specific traversability

Do not output only “ground” and “obstacle.” Compute whether the terrain is traversable for a specific vehicle.

A cell cost can depend on:

$[C = w_s(\text{slope}) + w_r(\text{roughness}) + w_h(\text{step height}) + w_c(\text{clearance}) + w_u(\text{uncertainty})]$

The thresholds should depend on:

- wheel radius,
- ground clearance,
- vehicle width,
- maximum climbing angle,
- and braking distance.

This turns the project from a visualization into a usable navigation component.

4.7 Weather-aware confidence, not “all-weather vision”

The report proposes intensity-based rain filtering.

Intensity alone is unreliable because it also changes with:

- range,
- material,
- angle of incidence,
- wetness,
- and sensor calibration.

Use a combined reliability score:

```
point_reliability =  
    local_spatial_consistency  
    + temporal_persistence  
    + expected_ring_density  
    + range-normalized_intensity  
    + return_type, if_available
```

For weather evaluation, SemanticSTF provides point-wise labels under rain, fog, and snow. IDD-3D is useful for Indian traffic and 3D detection/tracking, but it is not a direct replacement for a point-wise terrain segmentation dataset.

The correct monsoon claim is:

“When sensing quality degrades, the map increases uncertainty and prevents the planner from treating unobserved space as safe.”

Do not claim that the algorithm “sees through all weather.”

4.8 Graceful edge degradation

Create explicit operating levels:

Level	Behaviour
Full	Two surfaces, semantics, adaptive refinement, dynamics
Reduced	Two surfaces, fewer semantic classes, fewer patches
Geometry-only	Ground, obstacles, free/unknown; no semantic network
Emergency	Near-field collision map only

If latency exceeds the frame budget for several frames:

- reduce far-field resolution;
- disable optional semantics;
- preserve the near-field safety envelope;
- publish a quality/degradation indicator.

This is systems innovation that is highly feasible and impressive in a live demo.

5. Practical implementation strategy

Stage 1 — Build the mapping engine without ML

Use nuScenes-mini and its point-level labels as an **oracle semantic source**.

Build:

- one-sweep LiDAR loader;
- uniform grid baseline;
- ring/polar adaptive grid;
- elevation statistics;
- semantic voting;
- visualization;
- memory and latency measurement.

This follows the sensible “baseline first” advice already present in the guide.

This guarantees a working project even if ML integration fails.

Stage 2 — Add the strongest unique features

Prioritize:

1. occupied/free/unknown evidence;
2. two-surface cells;
3. path-corridor refinement;
4. terrain cost based on slope and roughness;
5. negative-obstacle detection.

These are more directly connected to mapping innovation than training a huge network.

Stage 3 — Add semantics

Prefer, in order:

1. existing point labels for an oracle upper bound;
2. a pretrained model;
3. lightweight range-image segmentation;
4. only then a sparse 3D model.

Do not begin by training LSK3DNet. The current proposal expects Patchwork++, LSK3DNet, TorchSparse++, TensorRT, ROS2, CUDA and Jetson integration simultaneously.

That is too risky unless the team already has those components running.

Stage 4 — Temporal behaviour

Add:

- ego-motion alignment;
- short frame history;
- static and dynamic layers;
- cell motion estimation;
- free-space-based clearing;
- time-based transition toward unknown.

Stage 5 — Planner integration

A basic A* planner is sufficient.

Use the output map to demonstrate:

- uniform low-resolution map misses a small hazard;
- uniform high-resolution map costs more memory/update time;
- distance-only adaptive map is efficient;
- risk-aware adaptive map preserves the hazard and generates a safer path.

6. Recommended dataset strategy

Purpose	Recommended data
Quick prototype	nuScenes-mini
Point semantic segmentation	nuScenes-lidarseg or SemanticKITTI
Off-road terrain	RELLIS-3D
Indian traffic objects	IDD-3D
Rain/fog semantic robustness	SemanticSTF
Controlled potholes/ditches	CARLA or custom synthetic point clouds
Multi-level/overhang demo	Small handcrafted synthetic scene

Do not force one dataset to prove everything. Use a **small benchmark suite**, with each dataset proving a specific claim.

A synthetic unit scene is especially valuable:

Scene A: small curb at 5 m
Scene B: pedestrian at 35 m in planned path
Scene C: branch above traversable road
Scene D: ditch / negative obstacle
Scene E: parked car versus moving car
Scene F: occluded area behind a vehicle

These controlled scenes make failures and improvements immediately understandable.

7. Evaluation framework

Core mapping metrics

- memory in actual bytes;
- cell count by resolution;
- average and 95th-percentile update latency;
- end-to-end FPS;
- quantization height error;
- occupancy precision and recall;
- free-space false-positive rate;
- percentage of correctly retained unknown/occluded cells.

Semantic metrics

- per-class IoU;
- macro mIoU;
- critical-class recall:
 - pedestrian,
 - vehicle,
 - curb,
 - pothole/ditch,
 - non-drivable terrain.

Do not report only mIoU. Large easy classes such as road and vegetation can hide poor performance on small safety-critical objects.

Multi-surface metrics

- overhead-clearance error;
- overhang false-obstacle rate;
- percentage of multilevel cells represented correctly;
- ground-surface height RMSE.

Temporal metrics

- ghost-trail duration;
- static/dynamic F1;
- map flicker rate;
- time needed to clear a moved object;
- persistence of a temporarily occluded static obstacle.

Planner-aware metrics

This should be one of the main innovations:

- collision rate;
- path success rate;
- minimum path clearance;
- path-length ratio;
- number of replans;
- planning time;
- weighted map error inside the planned corridor.

A slightly lower-mIoU map that produces safer paths is more useful than a visually cleaner map with dangerous local errors.

8. Essential baseline and ablation experiments

Compare:

1. uniform 5 cm grid;
2. uniform 50 cm grid;
3. distance-only ring/log-polar grid;
4. proposed risk-aware adaptive grid.

Then disable one component at a time:

Ablation	What it proves
No path-corridor refinement	Value of planner awareness
No uncertainty refinement	Value of active information allocation
One surface only	Benefit for branches/bridges/overhangs
No ray clearing	Effect on ghosts and occlusion safety
Semantic-only dynamics	Why class is not motion
No graceful degradation	Stability under computational load

This produces much stronger evidence than comparing only against OctoMap.

9. Suggested live demonstration

A memorable demo could proceed as follows:

1. Play a LiDAR sequence.
2. Show the raw cloud.
3. Show a uniform five-centimetre map with memory and update cost.
4. Switch to distance-only adaptive mode.
5. Enable risk-aware mode:
 - planned trajectory appears;
 - high-resolution patches move toward turns and hazards;

- a distant approaching object receives refinement.
6. Show a tree branch represented as an upper surface while the road remains traversable below.
 7. Show an occluded region marked unknown instead of green/free.
 8. Move a vehicle:
 - dynamic layer follows it;
 - free-space evidence clears its former location;
 - a parked vehicle remains stable.
 9. Run A* and compare path clearance.
 10. Artificially constrain computation and show graceful degradation without losing the near-field safety map.

The dashboard should display:

```
FPS
95th-percentile latency
active cells
memory usage
current quality level
occupied/free/unknown percentages
number of refined patches
minimum planned-path clearance
```

10. What to present as the main innovation

Use this concise pitch:

Existing adaptive LiDAR maps allocate resolution mainly according to distance. Our system allocates it according to navigation utility: planned-path proximity, time-to-collision, terrain complexity, temporal motion and uncertainty. Each adaptive cell retains multiple vertical surface hypotheses and explicit occupied/free/unknown evidence, allowing the map to represent overhangs, negative obstacles and occlusions without the memory cost of a full 3D voxel grid. The system also degrades gracefully on edge hardware while preserving a hard near-field safety envelope.

That is more original and more technically defensible than:

“We use log-polar coordinates and LSK3DNet.”

11. Recommended priorities

Must build

- LiDAR loader;
- uniform baseline;
- adaptive ring/polar map;
- cell height statistics;
- occupied/free/unknown state;
- real memory and latency measurements;
- interactive visualization.

Winning differentiators

- path- and TTC-aware refinement;
- two-surface cells;
- negative-obstacle detection;
- planner-aware evaluation;
- graceful degradation.

Optional

- pretrained semantic model;
- temporal cell velocity;
- weather reliability monitor;
- ROS2 output.

Avoid initially

- training LSK3DNet from scratch;
- custom CUDA kernels;
- TensorRT sparse-convolution conversion;
- full object tracking;
- complex camera fusion;
- claiming universal monsoon robustness;
- claiming log-polar mapping eliminates every artifact.

Final advice

The project should be framed primarily as an **intelligent mapping and resource-allocation system**, not as another semantic-segmentation project.

Many teams can attach a pretrained model to LiDAR. Fewer teams will show that their representation:

- knows where detail is valuable;
- understands what is unknown;
- handles overhangs and ditches;
- improves the planner's decisions;
- and remains operational under edge-compute or sensor degradation.

That combination is the strongest route to a distinctive and credible SIH submission.
